当代数据管理系统

大作业一（第一阶段）

基于 MongoDB 的在线问卷系统

项目完成报告

姓名：银启行

学号：10245501485

完成日期：2026 年 10 月 18 日

摘要

本项目完成了一个简化版的在线问卷系统，主要包括用户注册登录、问卷创建与管理、四类题目、条件跳转、答案校验、公开链接填写和结果统计。系统采用 Node.js + Express 作为后端，MongoDB 作为数据库，前端使用原生 HTML、CSS 和 JavaScript，没有使用现成问卷系统或前端框架。报告记录了系统结构、集合设计、跳转和校验的实现方法、AI 使用过程以及测试结果。目前 13 组接口测试全部通过，页面流程也在本机 MongoDB 6.0.19 上实际跑过。

这一阶段的目标不是把界面做得复杂，而是先把问卷数据怎么存、题目怎么校验、跳转怎么计算这几件事做清楚，为第二阶段的需求变更留出修改空间。

一、项目目标

大作业要求做一个类似问卷星或 Google Forms 的系统。注册用户可以创建问卷，设置题目和截止时间，发布一个可以发给别人的链接；其他登录用户通过链接填写，创建者可以查看统计结果。系统必须使用 MongoDB 保存业务数据，不能使用关系数据库，也不能直接套用现成问卷系统。

第一阶段我给自己定的目标有六项：

1. 完成注册、登录、退出，并保证用户只能看到自己的问卷。
2. 完成问卷的创建、编辑、发布、关闭和公开链接访问。
3. 支持单选、多选、文本填空、数字填空四种题型。
4. 把跳转规则保存在问卷数据中，填写时按规则自动跳题。
5. 在前后端都做校验，必答、多选数量、文本长度、数字范围和整数都要检查。
6. 按题目统计选项次数、回答人数、数字平均值和文本答案。

界面只做了课程作业需要的程度：白底、灰边框、普通按钮，能看清信息、能完成操作即可。重点放在数据库结构和后端逻辑上。

二、系统设计

2.1 总体结构

系统分为浏览器页面、Express 路由、业务模块和 MongoDB 四层。页面通过 fetch 调用 JSON 接口，登录状态用 session cookie 保存。题目校验、跳转路径和统计逻辑都放在后端，前端只负责显示和交互。

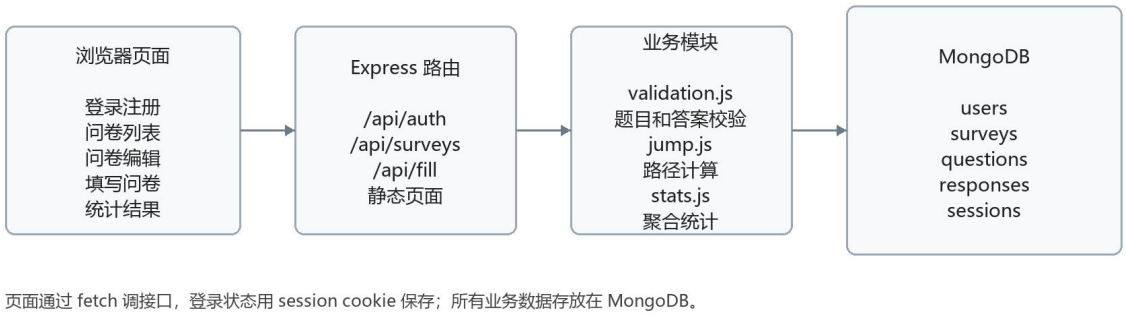


图 1 问卷系统结构

2.2 主要流程

用户第一次进入系统先注册账号，注册成功后直接登录。登录后进入“我的问卷”，这里显示自己创建的问卷，包括状态、题目数量、回收数量和截止时间。新建问卷后先进入草稿状态，在编辑页添加题目和跳转规则，确认无误后发布。发布后系统生成 8 位公开编号，链接形式为 /survey/xxxxxx。

填写者打开链接时会被要求先登录。填写页一次只显示一道题，点“下一题”后根据当前答案判断下一题题号。提交后答卷写到 responses 集合，创建者在统计页可以看到按题目汇总的结果。

2.3 问卷状态

问卷只有草稿、已发布、已关闭三种状态。截止时间不是单独的状态，而是在填写接口里实时判断。这样即使没有人手动关闭，过期后也不能继续填写。

状态	含义	可以做的操作
draft	草稿，还没有公开	改设置、加题、改题、发布、删除
published	已发布，链接可以访问	填写、看统计、关闭
closed	已关闭	看统计，不能再填写

2.4 页面实现

页面没有用 Vue 或 React，都是静态 HTML 加原生 JavaScript。这样代码量少，调试时也容易看清请求和返回值。问卷列表页把问卷按卡片排开，草稿和已发布问卷显示不同的按钮。管理页还会通过 /api/app-info 自动把 localhost 地址换成本机局域网 IP，方便手机填写。

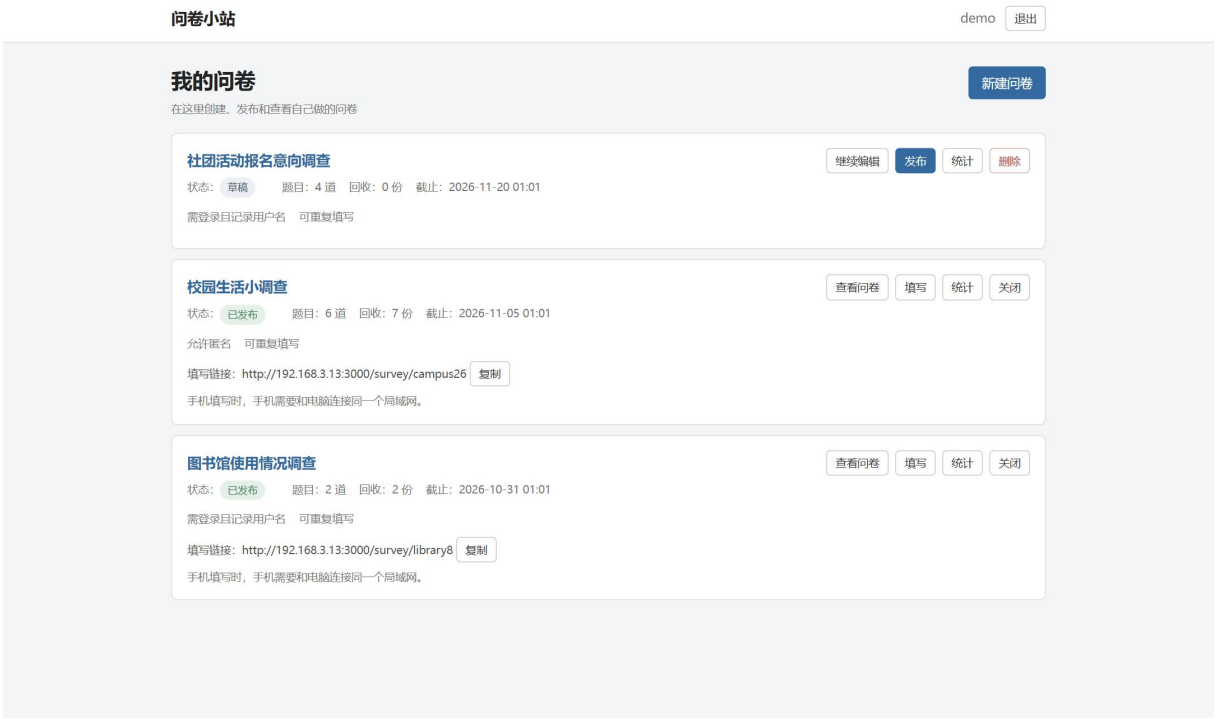


图 2 我的问卷页面

填写页顶部显示进度，当前只显示一道题。多选题下面会显示“请选择 2 至 3 项”这类提示，点上一题可以返回修改。返回后如果改了答案，原来后面的路径会被截断，避免把旧分支的答案带下去。

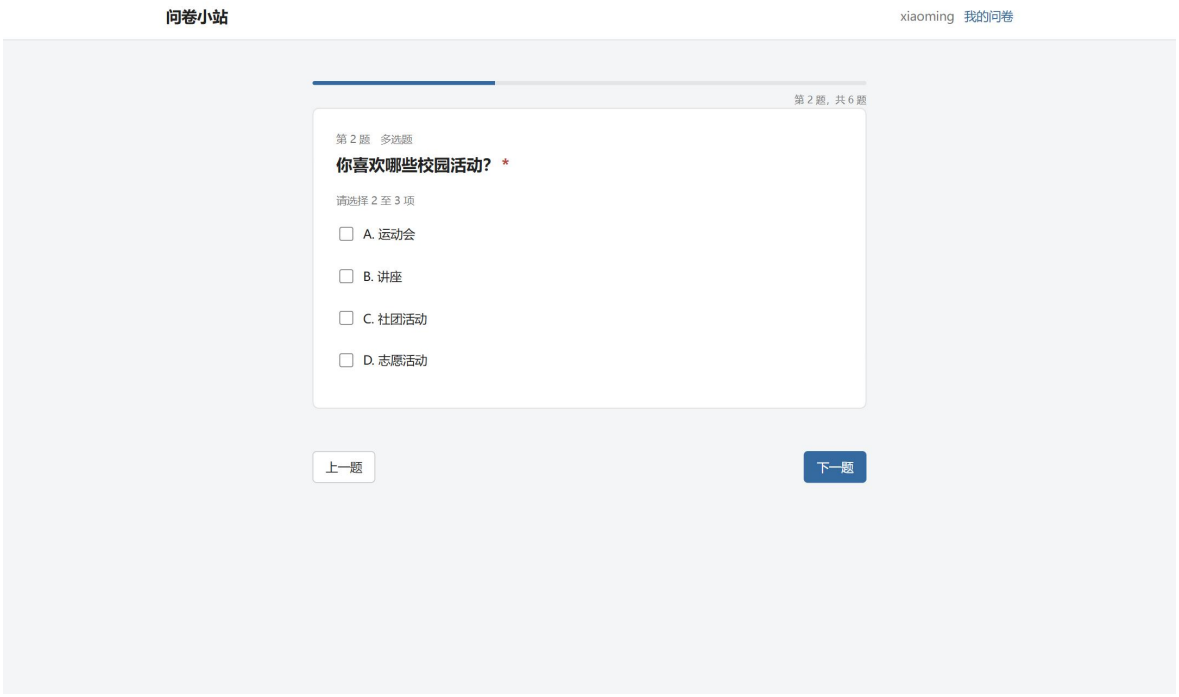


图 3 填写问卷页面

三、MongoDB 设计

3.1 集合划分

数据库名是 survey_system。业务数据分成 users、surveys、questions、responses 四个集合，登录 session 由 connect-mongo 放在 sessions 集合里。题目和答卷没有嵌在问卷文档里，这样问卷列表查询时不用读到大量无关内容。

集合	保存的内容	主要字段
users	用户账号	_id、username、password_salt、password_hash、created_at
surveys	问卷基本信息	_id、owner_id、public_id、title、status、deadline
questions	每道题目	_id、survey_id、order、type、options、jump_rules
responses	每次提交的答卷	_id、survey_id、respondent_id、answers、submitted_at
sessions	登录 session	connect-mongo 自动维护

3.2 主要文档结构

题目文档把同一道题需要的字段放在一起。选项用 key 和 text 两个字段，答案只保存 key，这样不依赖选项文字。跳转规则也放在题目文档的 jump_rules 数组里。

```
{
  "_id": ObjectId("66f20000000000000000101"),
  "survey_id": ObjectId("66f10000000000000000001"),
  "order": 1,
  "type": "single",
  "title": "你平时主要在哪里吃饭？",
  "required": true,
  "options": [
    { "key": "A", "text": "食堂" },
    { "key": "B", "text": "外卖" },
    { "key": "C", "text": "自己做" }
  ],
  "jump_rules": [
    { "condition": { "option": "A" }, "target_order": 4 },
    { "condition": { "option": "B" }, "target_order": 2 }
  ]
}
```

一份答卷把这次提交的答案放在 answers 数组里，只保存实际走到的题目：

```
{
  "survey_id": ObjectId("66f10000000000000000001"),
  "respondent_id": ObjectId("66f00000000000000000002"),
  "respondent_name": "xiaoming",
  "anonymous": false,
  "answers": [
    { "question_id": ObjectId("66f200...101"), "question_order": 1, "value": "A" },
    { "question_id": ObjectId("66f200...104"), "question_order": 4, "value": 20 },
    { "question_id": ObjectId("66f200...106"), "question_order": 6, "value": 8 }
  ],
  "submitted_at": ISODate("2026-10-15T01:20:00Z")
}
```

3.3 集合关系

用户和问卷是一对多，问卷和题目、答卷都是一对多。answers 是答卷内部的嵌套数组，jump_rules 是题目内部的嵌套数组。

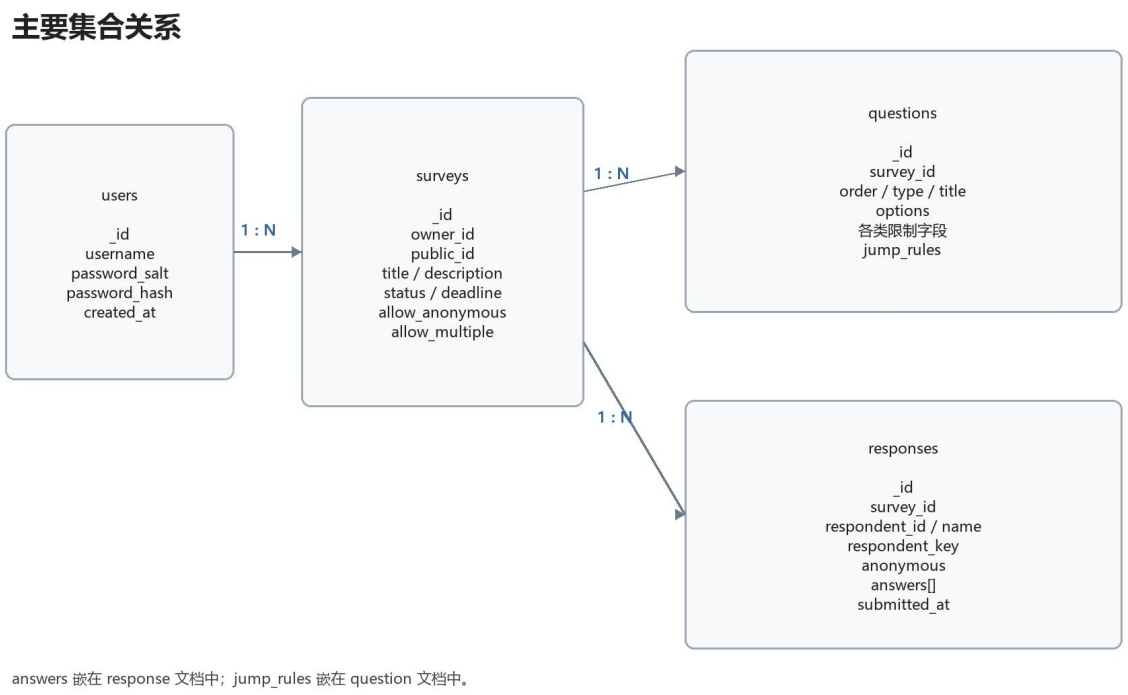


图 4 主要集合关系

3.4 索引

集合	索引	用途
users	{ username: 1 } unique	防止重名，登录时快速查用户
surveys	{ public_id: 1 } unique	根据公开链接找问卷
surveys	{ owner_id: 1, created_at: -1 }	查询某个用户的问卷列表
questions	{ survey_id: 1, order: 1 } unique	按问卷取题，保证题号不重复
responses	{ survey_id: 1, submitted_at: -1 }	取答卷和做统计
responses	{ survey_id: 1, respondent_key: 1 }	限制重复填写时判重

3.5 设计理由

最开始我想把题目直接放在 survey 文档里，因为看起来最省事。但这样每加一道题都要把整个问卷读出来再写回去，问卷多了以后列表查询也会读到一堆不需要的题目。后来改成 questions 集合，一道题一个文档，题号 order 单独保存，添加和修改都只动一条记录。

答卷里的 answers 仍然用嵌套数组。一份答卷是一个整体，提交后一般不会单独改某道题，统计时按 survey_id 找到答卷，再展开 answers 就可以，不需要再拆一个答案集合。

没有用关系数据库，主要是因为四种题型的字段不一样。选择题有选项和选择数量，文本题有字数限制，数字题有范围限制，跳转规则又只出现在部分题型上。用关系表要拆题目表、选项表、规则表，还要处理很多可空字段和连接查询。MongoDB 把一道题完整放在一个文档里，和前后端传的 JSON 也对得上。

系统里有少量冗余：question_count 是题目数量缓存，respondent_name 是提交时的用户名快照，answers.question_order 是提交时的题号。它们只用于显示和排序，不参与权限判断。题目发布后不允许修改，所以题号和答案不会对不上。

四、跳转设计

4.1 规则的保存方式

跳转规则没有写在程序里的 if/else 中，而是存在题目的 jump_rules 数组里。单选题按选项判断，多选题判断答案数组里有没有某个选项，数字题按比较符判断。规则从上到下匹配，第一条满足的生效。

```
单选题：{ condition: { option: "A" }, target_order: 4 }
多选题：{ condition: { option: "D" }, target_order: 6 }
数字题：{ condition: { op: "gte", value: 30 }, target_order: 6 }
```

保存规则时会检查目标题号必须大于当前题号，并且不能超过题目总数。这样正常编辑时不会做出往回跳或循环的规则。

4.2 路径计算

填写页和后端使用同一套规则。后端计算路径的伪代码如下：

```
order = 1
path = []
visited = set()
while order 存在 and order not in visited:
    path.append(order); visited.add(order)
    question = questions[order]
    answer = answers[order]
    rule = question.jump_rules 中第一条匹配 answer 的规则
    order = rule.target_order if rule else order + 1
```

前端计算路径只是为了显示下一题，后端提交时还会重新算一遍。这样即使有人绕过页面直接发请求，也不能提交一条和规则不一致的答卷。后端只会校验路径上的题目，跳过的题即使传了答案也会被忽略。

4.3 实际例子

演示问卷第 1 题选 A 跳到第 4 题，选 B 跳到第 2 题；第 2 题选中 D 跳到第 6 题；第 4 题填大于等于 30 跳到第 6 题。三条填写路径如下：

回答情况	实际经过的题号	保存的答案
第 1 题 A，第 4 题 25	1 → 4 → 5 → 6	1、4、6（第 5 题选答且为空）
第 1 题 B，第 2 题 A+B，第 4 题 40	1 → 2 → 3 → 4 → 6	1、2、3、4、6
第 1 题 B，第 2 题 A+D	1 → 2 → 6	1、2、6

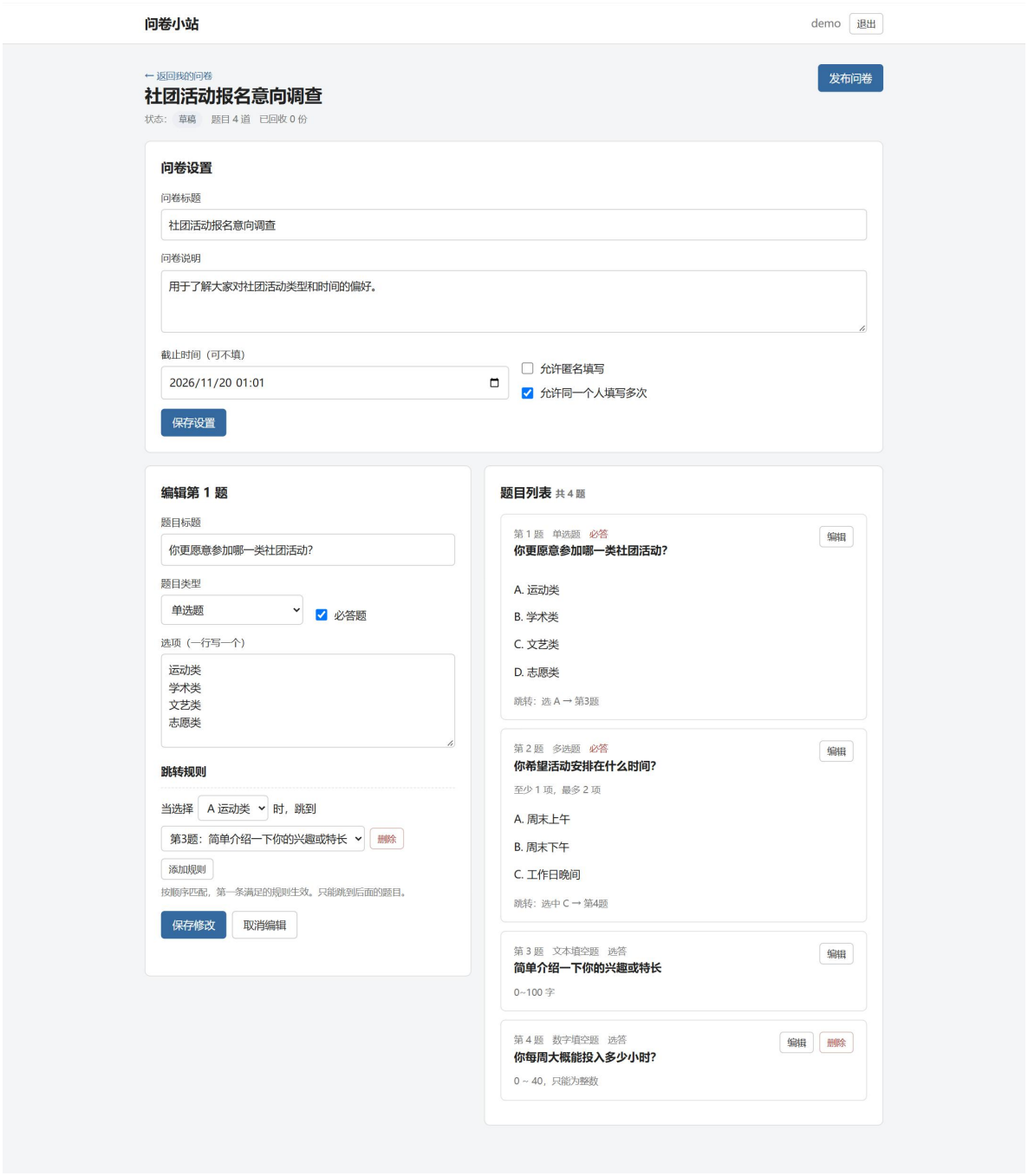


图 5 问卷编辑和跳转规则设置

编辑页左边是题目表单，右边是题目列表。选中已有题目后，表单会带出原来的选项和跳转规则；规则行可以增加和删除，目标题只列出当前题后面的题目，避免误选到前面。

五、校验设计

5.1 题型校验

校验分前端和后端两遍。前端在点下一题时提示，后端在提交接口里再检查一次，以后端为准。

题型	检查内容	提示示例
单选题	必答时不能为空，答案必须是已有选项	本题为必答题，请填写后再继续
多选题	最少、最多或必须选择数量	本题至少选择 2 项
文本题	最少字数、最多字数	至少需要填写 5 个字
数字题	是否为数字、是否为整数、最小值和最大值	本题只能填写整数

5.2 服务端校验顺序

1. 检查问卷是否存在、是否已发布、是否已关闭、是否超过截止时间。
2. 检查用户是否登录，以及当前问卷是否允许重复填写。
3. 按跳转规则算出本次答卷实际经过的题目。
4. 逐题检查是否为空、是否必答、答案格式是否正确。
5. 只把实际路径上的答案整理后写入 responses。

5.3 匿名和重复填写

匿名答卷不保存 respondent_id 和 respondent_name。但如果问卷限制每人只能填一次，又需要判断同一个人有没有填过。我的做法是保存一个 HMAC 值 respondent_key，由用户编号和问卷编号一起算出来，只用于查询是否已经提交。创建者在页面上看不到这个值，也不能直接用它找到具体用户。

另外，问卷发布后题目和基本设置会锁定。这样已经收到的答案仍然对应原来的题目，不会因为后来改题导致统计错位。

六、API 说明

接口都放在 /api 下面，返回 JSON。未登录访问管理接口返回 401，不是问卷创建者返回 404，答案校验不通过返回 400，重复填写返回 409。下面只列主要接口，完整字段和示例见 docs/API 说明.md。

方法	路径	说明
GET	/api/app-info	获取局域网或公网分享地址
POST	/api/auth/register	注册并自动登录
POST	/api/auth/login	登录
GET	/api/surveys	当前用户的问卷列表
POST	/api/surveys	新建问卷
GET	/api/surveys/:id	问卷详情和题目
PATCH	/api/surveys/:id	修改草稿设置
POST	/api/surveys/:id/questions	添加题目
PATCH	/api/surveys/:id/questions/:qid	修改题目和跳转
POST	/api/surveys/:id/publish	发布问卷
POST	/api/surveys/:id/close	关闭问卷
GET	/api/surveys/:id/stats	统计结果
GET	/api/fill/:publicId	获取可填写的问卷
POST	/api/fill/:publicId	提交答卷

提交答卷的请求示例：

```
{
  "anonymous": true,
  "answers": [
    { "question_order": 1, "value": "A" },
    { "question_order": 4, "value": 20 },
    { "question_order": 6, "value": 8 }
  ]
}
```

统计接口返回 total_responses 和每道题的统计。单选题返回每个选项的 count 和 percent，多选题的 percent 按回答本题的人数计算，数字题返回 average、min、max，文本题返回 values 数组。

七、AI 使用过程

7.1 使用日志

开发时主要用 ChatGPT（Codex）帮忙写初稿、查聚合写法和检查测试。我的做法是把需求拆成小问题，每次只问一部分，拿到代码后在本地运行，再根据报错和页面效果修改。下面是比较有代表性的几次记录。

阶段	提示词（大意）	AI 给出的结果	我做的修改
数据库	用 MongoDB 设计问卷系统，支持四种题型和跳转	建议把题目嵌在 survey 里	拆出 questions 和 responses 集合
账号	写 Express 注册登录，密码不要存明文	给了 scrypt 和 session 代码	session 改成存 MongoDB
跳转	实现单选、多选、数字题的跳转，不要写死	只处理了单选，用 if 判断	改成 jump_rules，写路径重算
校验	写多选题至少、最多、必须选择的校验	只写了必答和最少数量	补全最多、必须、文本和数字校验
统计	统计多选每个选项被选次数	只按数组分组，结果不对	改成两次 unwind 再 group
测试	写接口测试覆盖创建、提交和统计	没有保存 cookie，登录后全是 401	写了带 cookie 的测试客户端
匿名	匿名问卷怎么限制重复填写	建议匿名答卷也存用户 id	改成 HMAC 的 respondent_key
页面	做一个问卷编辑和填写页面	第一版颜色和卡片比较多	简化成普通的白底页面

7.2 总结

AI 帮我省时间的地方主要是项目骨架、路由、CSS、聚合管道写法和测试场景。但 AI 第一版经常只考虑主流程：跳转只做单选，校验只做必答，匿名和判重也会互相矛盾。这些问题如果直接复制代码，看起来能跑，实际测试时会出问题。

我自己做的主要工作是确定集合怎么拆、跳转规则用什么结构、匿名怎么判重、前端返回上一题时怎么处理旧路径，以及把每段代码放到真实 MongoDB 上跑一遍。测试里的断言、演示数据和最后的报告内容也是按实际结果改的，没有把 AI 生成的示例结果直接写进报告。

遇到问题时我会先看浏览器 Network 里的状态码和 error，再看服务端控制台，数据不对就直接用 mongosh 查集合。这样比继续问 AI 更快，也更容易发现字段名或类型写错。

八、测试结果

8.1 测试环境和方法

测试使用 Node.js 自带的 node:test，不依赖额外的测试框架。每次测试启动一个真实的 Express 服务，连接一个带时间戳的 MongoDB 测试库，通过 HTTP 请求走完整接口流程，跑完后删除测试库。

```
cd survey-mongodb
npm test
```

8.2 主要测试用例

编号	测试内容	输入或操作	预期结果	结果
T00	局域网分享地址	从 localhost 请求 app-info	返回局域网 IP，不含 localhost	通过
T01	注册和重复注册	alice / 123456，再注册同名	第一次 201，第二次 409	通过
T02	创建问卷和四种题	添加 6 道题	题号 1-6，question_count=6	通过
T03	跳转规则保存	第 1 题 A→4、B→2；第 2 题 D→6	保存成功，发布成功	通过
T04	公开字段	另一个用户读取填写页	没有 owner_id 等内部字段	通过
T05	必答和格式校验	空答案、abc、25.5、2 个字	分别返回 400	通过
T06	单选跳转	第 1 题 A，填第 4、6 题	保存题号 1、4、6	通过
T07	多选跳转	第 1 题 B，第 2 题 A+D	保存题号 1、2、6	通过
T08	顺序和数字跳转	第 4 题 40，跳过第 5 题	保存题号 1、2、3、4、6	通过
T09	统计	3 份答卷	单选、多选、数字、文本结果正确	通过
T10	用户隔离	bob 查 alice 的问卷	返回 404	通过
T11	匿名限填	同一用户匿名提交两次	第一次 201，第二次 409	通过
T12	关闭和截止	关闭后再填；过期问卷发布	分别返回 403 和 400	通过

8.3 页面检查

除了接口测试，我还用演示账号走了一遍页面：注册登录、新建问卷、编辑第 1 题、发布、用另一个账号填写、查看统计、关闭问卷。统计页显示 7 份答卷，第 1 题食堂 3 人、外卖 3 人、自己做 1 人，第 4 题平均 37.5 分钟，和第 3 章里的数据对得上。

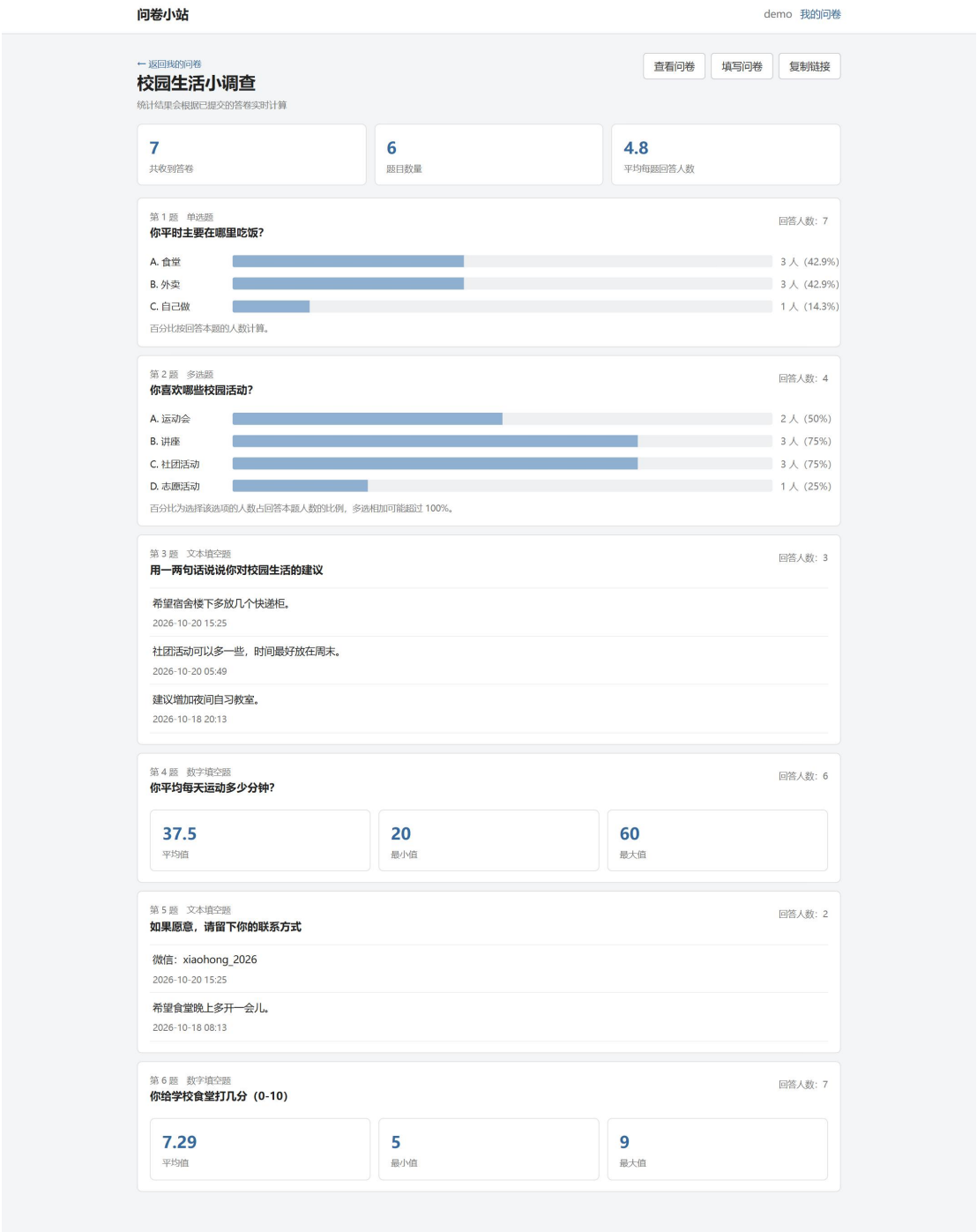


图 6 统计结果页面

8.4 实际运行结果

npm test 的最终输出如下：

```
✔ 自动生成局域网分享地址
✔ 注册、登录和用户信息
✔ 创建问卷并添加四种题型
✔ 设置单选题、多选题和数字题的跳转规则
✔ 填写页面只能读到公开字段
✔ 必答、多选数量、文本长度和数字范围校验
✔ 三种跳转路径都能提交并正确保存
✔ 统计结果按题目分别计算
✔ 用户只能看到自己的问卷
✔ 匿名、限填一次和重复填写限制
✔ 关闭问卷后不能再填写，统计仍然可看
✔ 截止时间已过的问卷不能发布

tests 13    pass 13    fail 0
```

九、遇到的问题与解决

9.1 题目放在哪里

最开始把题目放在 survey 文档里，写添加题目接口时发现每次都要读出整个问卷再写回，代码反而更绕。后来拆成 questions 集合，用 survey_id 关联，问题就简单了。代价是统计时要先取题目再逐题聚合，但数据量不大，几道题的统计速度可以接受。

9.2 跳转规则容易写死

第一版跳转是前端写死的 if，只支持单选题，而且改一道题就要改代码。后来把规则保存到 jump_rules，用统一的匹配函数处理单选、多选和数字题，页面只负责把规则显示出来。

9.3 匿名和判重的矛盾

如果匿名答卷保存用户 id，就不算真正匿名；如果不保存，又没法限制每人填一次。最后用 HMAC 生成 respondent_key，只存这个值用于判重，不存用户 id 和用户名。

9.4 多选题统计不对

多选题的 value 是数组，一开始直接按数组分组，结果每个不同组合各算一项。后来先展开 answers，再展开 value 数组，最后按单个选项 key 分组，才得到每个选项被选了多少次。

9.5 测试登录状态丢失

接口测试第一版每个请求都重新 fetch，没有把 set-cookie 带到下一个请求，注册完再请求就是 401。后来写了一个带 cookie 的小客户端，在每个请求里手动带 Cookie 头，测试才跑通。

9.6 测试数据把跳过题也填了

做演示数据时，有一份答卷在第 2 题选了会跳到第 6 题的选项，但我又把第 3、4 题的答案放了进去。统计看起来正常，实际上路径不合法。发现后按跳转规则重新整理了演示答卷，并在测试里专门断言保存的题号顺序。

9.7 页面一开始太像商业产品

AI 第一版页面用了渐变、卡片阴影和图标，看起来不像课程作业，也不方便改。后来统一成白底、灰边框、一个蓝色主按钮，信息保留，装饰去掉，页面反而更好检查。

十、总结和不足

这一阶段完成了问卷系统的核心流程，数据全部存在 MongoDB 中，四种题型、跳转、校验、匿名、统计和权限都有测试覆盖。通过这个项目，我对文档模型和关系模型的区别有了更具体的感受：不是所有数据都要拆表，也不是所有东西都适合塞进一个文档，要看读写方式和数据量。

系统还有不足。发布后不能改题，虽然保证了一致性，但用户想改错别字时只能重新建问卷；题目不能拖动排序；统计只有文字和简单的横条，没有图表和导出；重复填写的判重是先查再插，极端并发下仍可能插入两份，要彻底解决需要唯一索引或事务。

第二阶段如果需求变化，我会在这个结构上扩展：题目仍然单独存，跳转规则继续用数据配置，旧答卷保留，新增字段尽量给默认值，不删除第一阶段的功能。这样修改成本会比重新做一个系统低。

附录：运行方法和文件说明

本机运行步骤：

```
cd survey-mongodb
npm install
npm run seed      # 可选，写入演示账号和问卷
npm start
# 浏览器打开 http://localhost:3000
```

演示账号：demo / demo123（查看管理和统计），xiaoming / 123456（填写问卷）。主要代码在 app.js、src/routes、src/validation.js、src/jump.js、src/stats.js，页面在 public 目录，测试在 test/api.test.js。

系统说明、数据库设计、API 说明、关键逻辑、测试用例和 AI 使用日志分别放在 docs 目录下，页面截图放在 output/screenshots 目录下。